MAE 3403 HW 2 — Highlighted Code Changes (Unified Diff)

Green = added lines, Red = removed lines, Gray = diff headers, Black = context

Generated: 2026-02-16 03:12:59

Gauss_Elim.py

```diff
--- Gauss_Elim.py (original)
+++ Gauss_Elim.py (edited)
@@ -29,9 +29,28 @@
     :param A: The matrix to sort
     :return: The sorted matrix
     """
-    pass
-
-# region row operations
+    # Make a deep copy so we do not modify the original matrix
+    AA = dc(A)
+    n_rows = len(AA)
+    n_cols = len(AA[0])
+
+    # Greedy reordering: for each diagonal index i, pick the remaining row
+    # with the largest |A[row][i]| and swap it into position i.
+    for i in range(min(n_rows, n_cols)):
+        r_best = i
+        best_val = abs(AA[i][i])
+        for r in range(i + 1, n_rows):
+            v = abs(AA[r][i])
+            if v > best_val:
+                best_val = v
+                r_best = r
+        if r_best != i:
+            AA = SwapRows(AA, i, r_best)
+
+    # NOTE: This routine improves diagonal dominance when possible, but does not
+    # guarantee strict diagonal dominance for all matrices.
+    return AA
+
 def SwapRows(A, r1, r2):  #if I do this, it is known as a partial pivot
     '''
     One of the elementary row operations in Gaussian elimination.
```

NumericalMethods.py

```diff
--- NumericalMethods.py (original)
+++ NumericalMethods.py (edited)
@@ -20,9 +20,28 @@
     :return: probability value
     """
     mu, sig = args
-    p = Simpson(PDF, (mu, sig, mu - 5 * sig, 0))
+    lhl = mu - 5.0 * sig
+    rhl = mu + 5.0 * sig
+
+    # If we want P(x < c), integrate from lhl to c.
+    # If we want P(x > c), integrate from c to rhl.
+    if GT:
+        a = c
+        b = rhl
+    else:
+        a = lhl
+        b = c
+
+    # Clamp integration bounds to [lhl, rhl]
+    if a < lhl:
+        a = lhl
+    if b > rhl:
+        b = rhl
+    if b < a:
+        return 0.0
```

```diff
+
+    p = Simpson(PDF, (mu, sig, a, b), N=1000)
     return p
-
 def GPDF(args):
     """
     Here is where I will define the Gaussian probability density function.
@@ -54,9 +73,27 @@
     :param args: a tuple containing (mean, stDev, lhl, rhl)
     :return: the area beneath the function between lhl and rhl
     """
-    area = 0.5
+    mu, sig, lhl, rhl = args
+
+    # Simpson's 1/3 rule requires an even number of subintervals
+    if N % 2 == 1:
+        N += 1
+    a = float(lhl)
+    b = float(rhl)
+    h = (b - a) / N
+
+    s = fn((a, mu, sig)) + fn((b, mu, sig))
+
+    for i in range(1, N, 2):
+        x = a + i * h
+        s += 4.0 * fn((x, mu, sig))
+
+    for i in range(2, N, 2):
+        x = a + i * h
+        s += 2.0 * fn((x, mu, sig))
+
+    area = (h / 3.0) * s
     return area
-
 def Secant(fcn, x0, x1, maxiter=10, xtol=1e-5):
     """
     This funciton implements th Secant method to find the root of an equation.  You should write your equatio
n in a form
@@ -68,8 +105,25 @@
     :param xtol:  exit if the |xnewest - xprevious| < xtol
     :return: tuple with: (the final estimate of the root (most recent value of x), number of iterations)
     """
-    pass
+    f0 = fcn(x0)
+    f1 = fcn(x1)
+    iters = 0

+    for   in range(maxiter):
+        denom = (f1 - f0)
+        if denom == 0:
+            return x1, iters
+
+        x2 = x1 - f1 * (x1 - x0) / denom
+        iters += 1
+
+        if abs(x2 - x1) < xtol:
+            return x2, iters
+
+        x0, x1 = x1, x2
+        f0, f1 = f1, fcn(x1)
+
+    return x1, iters
 def GaussSeidel(Aaug, x, Niter = 15):
     """
     This should implement the Gauss-Seidel method (see page 860, Tabl 20.2) for solving a system of equations
.
@@ -78,25 +132,25 @@
     :param Niter:  Number of iterations to run the GS method
     :return: the solution vector x
     """
```

```
-    # Step 1:  make the augmented matrix diagonal dominant
-    # Step 2:  in a loop:
-    # Step 2a:  solve first row for x[0] using old values for x[1], etc
-    # Step 2b:  solve remaining rows for x[i] using new values above and old values below
-    # Step 3:  return x after Niter
+    # Step 1: try to reorder rows to improve diagonal dominance
     Aaug = GE.MakeDiagDom(Aaug)
-    n_Rows = len(Aaug)
-    n_Cols = len(Aaug[0])
+    n_rows = len(Aaug)
+    n_cols = len(Aaug[0])  # = n_rows + 1
```

```
-    for i in range(Niter):
-        for i in range(len(n_Rows)):
-            rhs = Aaug[i][n_Cols-1]
-            for j in range(i-1,0,-1):
-                rhs -= Aaug[i][j]*x[j]
-            for j in range(i+1,n_Cols-1):
-                rhs -= Aaug[i][j]*x[j]
-            x[i]=rhs/Aaug[i][i] # we can run into a problem if the diagonal has a zero
+    for   in range(Niter):
+        for i in range(n_rows):
+            b_i = Aaug[i][n_cols - 1]
+            s = 0.0
+            for j in range(n_cols - 1):
+                if j != i:
+                    s += Aaug[i][j] * x[j]
+
+            a_ii = Aaug[i][i]
+            if a_ii == 0:
+                continue
+            x[i] = (b_i - s) / a_ii
+
     return x
-
 def main():
     '''
     This is a function I created for testing the numerical methods locally.
```

hw2a.py

```
--- hw2a.py (original)
+++ hw2a.py (edited)
@@ -1,29 +1,22 @@
 #region imports
-from math import sqrt, pi, exp
-from NumericalMethods import GPDF, Simpson, Probability
+from NumericalMethods import GPDF, Probability
 #endregion

 #region function definitions
 def main():
-    """
-    I want to integrate the Gaussian probability density function between
-    a left hand limit = (mean - 5*stDev) to a right hand limit = (c).  Here
-    is my step-by-step plan:
-    1. Decide mean, stDev, and c and if I want P(x>c) or P(x<c).
-    2. Define args tuple and c to be passed to Probability
-    3. Pass args, and a callback function (GPDF) to Probability
-    4. In probability, pass along GPDF to Simpson along with the appropriate args tuple
-    5. Return the required probability from Probability and print to screen.
-    :return: Nothing to return, just print results to screen.
-    """
-    #region testing user input
-    # The following code solicites user input through the CLI.
-    mean = input("Population mean? ")
-    stDev = input("Standard deviation?")
-    c = input("c value?")
-    GT = True if input("Probability greater than c?").lower() in ["y","yes","true"] else "False"
-    print("P(x"+(">" if GT else "<") + c +"|"+mean+", "+stDev +")")
-    #endregion
```

```
+    """Run part (a): compute required Gaussian probabilities and print to CLI."""
+
+    # 1) P(x < 105 | N(100, 12.5))
+    mu1, sig1, c1 = 100.0, 12.5, 105.0
+    p1 = Probability(GPDF, (mu1, sig1), c1, GT=False)
+
+    # 2) P(x > mu + 2*sigma | N(100, 3))
+    mu2, sig2 = 100.0, 3.0
+    c2 = mu2 + 2.0 * sig2  # 106.0
+    p2 = Probability(GPDF, (mu2, sig2), c2, GT=True)
+
+    print(f"P(x<{c1:0.2f}|N({mu1:0.0f},{sig1:0.1f}))={p1:0.2f}")
+    print(f"P(x>{c2:0.2f}|N({mu2:0.0f},{sig2:0.0f}))={p2:0.2f}")
 #endregion

 if __name__ == "__main__":
```

hw2b.py

```
--- hw2b.py (original)
+++ hw2b.py (edited)
@@ -1,29 +1,30 @@
 #region imports
+from math import cos
 from NumericalMethods import Secant
 #endregion

 #region function definitions
 def fn1(x):
-    pass
+    """Return f(x) = x - 3*cos(x). Root satisfies x = 3*cos(x)."""
+    return x - 3.0 * cos(x)

 def fn2(x):
-    pass
+    """Return f(x) = cos(2x) * x^3. Root occurs when cos(2x)=0 or x=0."""
+    return cos(2.0 * x) * (x ** 3)

 def main():
-    """
-       fn1:  x-3cos(x)=0; with x0=1, x1= 2, maxiter = 5 and xtol = 1e-4
-       fn2:  cos(2x)*x**3; with x0=1, x1= 2, maxiter = 15 and xtol = 1e-8
-       fn2:   with x0=1, x1= 2, maxiter = 3 and xtol = 1e-8
+    """Run part (b): compute roots with the Secant method and print to CLI."""

-       I observe that for functions 2 and 3, the answer should be pi/2 or about 1.57
-    :return: nothing, just print results
-    """
-    r1 = Secant(fn1, 1, 2, 5,1e-4)
-    r2 = Secant(fn2, 1,2,15, 1e-8)
-    r3 = Secant(fn2,1,2,3,1e-8)
-    print("root of fn1 = {root:0.4f}, after {iter :0d} iterations".format(root=r1[0], iter=r1[1]))
-    #etc.
-    pass
+    r1 = Secant(fn1, 1.0, 2.0, maxiter=5, xtol=1e-4)
+    r2 = Secant(fn2, 1.0, 2.0, maxiter=15, xtol=1e-8)
+    r3 = Secant(fn2, 1.0, 2.0, maxiter=3, xtol=1e-8)
+
+    print("fn1: x - 3cos(x) = 0")
+    print("  root = {root:0.6f}, iterations = {it:d}".format(root=r1[0], it=r1[1]))
+
+    print("fn2: cos(2x)*x^3 = 0")
+    print("  root = {root:0.10f}, iterations = {it:d} (maxiter=15)".format(root=r2[0], it=r2[1]))
+    print("  root = {root:0.10f}, iterations = {it:d} (maxiter=3)".format(root=r3[0], it=r3[1]))
 #endregion

 if __name__=="__main__":
```

hw2c.py

```diff
--- hw2c.py (original)
+++ hw2c.py (edited)
@@ -1,11 +1,32 @@
 # region imports
-from copy import deepcopy
 from NumericalMethods import GaussSeidel
-from Gauss_Elim import AugmentMatrix
 # endregion
+
 def main():
-    pass
+    """Run part (c): solve two linear systems using Gauss-Seidel."""

+    # System 1:
+    Aaug1 = [
+        [3.0, 1.0, 1.0, 2.0],
+        [1.0, 4.0, 1.0, 12.0],
+        [2.0, 1.0, 2.0, 10.0],
+    ]
+    x0_1 = [0.0, 0.0, 0.0]
+    x1 = GaussSeidel(Aaug1, x0_1, Niter=15)
+    print("System 1 solution (Gauss-Seidel, 15 iter):")
+    print([round(v, 6) for v in x1])
+
+    # System 2:
+    Aaug2 = [
+        [1.0, 10.0, 2.0, 4.0, 2.0],
+        [3.0, 1.0, 4.0, 12.0, 12.0],
+        [9.0, 2.0, 3.0, 4.0, 21.0],
+        [1.0, 2.0, 7.0, 3.0, 37.0],
+    ]
+    x0_2 = [0.0, 0.0, 0.0, 0.0]
+    x2 = GaussSeidel(Aaug2, x0_2, Niter=15)
+    print("System 2 solution (Gauss-Seidel, 15 iter):")
+    print([round(v, 6) for v in x2])

 if __name__=="__main__":
     main()
```